

CCDV-F Exam Cheat Sheet

Skills measured per Exam Guide v1.0, effective July 2026.

Exam Weights

Domain	%	Focus
Agents and Workflows	14.7%	Workflow vs. agent, Agent SDK, hooks, subagents, frameworks
Applications and Integration	33.1%	Requirements, API mechanics, app design, config management
Claude Code	3.1%	CLAUDE.md, settings.json, session modes
Eval, Testing, and Debugging	2.6%	Error types, trace analysis, failure isolation
Model Selection and Optimization	16.8%	Tokens, sampling, model tiers, reasoning modes, cost
Prompt and Context Engineering	11.0%	Diagnostic prompting, context management, output handling
Security and Safety	8.1%	Prompt injection, guardrails, hooks, key management
Tools and MCPs	10.6%	Tool implementation, MCP servers, agentic customization

Applications and Integration alone is a third of the exam — don't under-invest there relative to the flashier agent/security content.

Decision Trees

Workflow vs. agent

Can you map out the exact steps in advance?	→ Workflow (predictable code path)
Steps can't be predicted; needs model-driven judgment?	→ Agent (model directs its own process)
Not sure?	→ Start with an agent, extract workflow patterns as they emerge from real usage

The five named workflow patterns

Fixed subtasks, decomposition improves focus?	→ Prompt chaining
Distinct input categories need different handling?	→ Routing
Independent subtasks, or need multiple attempts?	→ Parallelization (sectioning / voting)
Subtask structure varies by input, can't predict?	→ Orchestrator-workers
Clear eval criteria, refinement improves output?	→ Evaluator-optimizer

Built-in tool vs. custom tool vs. Skill vs. MCP server

Generic capability (file I/O, shell, web)?	→ Built-in tool
One app, one specific function, no reuse elsewhere needed?	→ Custom tool
Repeatable process/judgment call, no new live data access needed?	→ Skill
Multiple apps need the same live/dynamic data or action, maintained independently of any one app?	→ MCP server
Remember: MCP connects Claude to data; Skills teach it what to do with that data.	

Realtime vs. batch API

User/process waiting on result now?	→ Synchronous or streaming (realtime)
Latency-tolerant, high-volume, cost matters more than turnaround (up to 24h OK)?	→ Message Batches API (lower per-token cost)

Model tier selection

Default starting point	→ Sonnet 5 (best speed/intelligence balance)
Eval shows Sonnet missing the quality bar	→ move up to Opus 5 (complex agentic/enterprise)
Need the highest available capability adaptive thinking)	→ Fable 5 (long-running agents, always-on
Eval shows quality drop is acceptable	→ move down to Haiku 4.5 (fastest, cheapest, extended thinking not adaptive)

Prompt failure → missing technique

Wrong output SHAPE (prose instead of a label/JSON)	→ Output constraint
Content drifts / scope creeps across turns	→ System prompt (or a more specific one)
Right task, invented structure	→ Few-shot examples
Clean on tested inputs, breaks on a variant	→ Constraint covering that variant
Re-prompted 5x and still wrong?	→ Stop. Diagnose the failure type
first.	

Pruning vs. compaction (context bloat)

Bloat is re-fetchable tool output (big file read, verbose command)?	→ Prune/clear (cheap, lossless)
Bloat is dialogue/reasoning that can't be cheaply re-fetched?	→ Compaction (LLM summarize, costs more)

Claude API Error Codes

Status	Type	Retryable?
400	<code>invalid_request_error</code>	No — fix request
401	<code>authentication_error</code>	No
402	<code>billing_error</code>	No
403	<code>permission_error</code>	No
413	<code>request_too_large</code>	No — reduce payload
429	<code>rate_limit_error</code>	Yes — honor <code>retry-after</code> , backoff
529	<code>overloaded_error</code>	Yes — exponential backoff

Agent SDK loop-level (`ResultMessage.subtype`, distinct from HTTP errors): `success`, `error_max_turns`, `error_max_budget_usd`, `error_during_execution`, `error_max_structured_output_retries`.

Model Lineup Quick Reference

Tier	Context	Reasoning mode	Latency	Positioning
Fable 5	1M	Adaptive (always on)	Slower	Most capable; long-running agents

Tier	Context	Reasoning mode	Latency	Positioning
Opus 5	1M	Adaptive	Moderate	Complex agentic coding, enterprise
Sonnet 5	1M	Adaptive	Fast	Default balance
Haiku 4.5	200K	Extended (not adaptive)	Fastest	Cheapest, near-frontier speed

Gotcha: Haiku 4.5 is the odd one out — extended thinking, not adaptive. `budget_tokens` is deprecated and 400s on newest models; effort levels (`low` / `medium` / `high` / `xhigh` / `max`) replace it for adaptive thinking.

Prompt Caching

	Automatic	Explicit
Config	One top-level <code>cache_control</code>	<code>cache_control</code> per content block
Breakpoints	System slides forward automatically	Up to 4 per request
TTL	5-min default	5-min or 1-hour (higher write cost)

Always subject to: minimum token threshold per block, 20-block lookback window.

Settings.json Precedence

```
Managed (org IT, cannot override) > CLI flags > Local (.claude/settings.local.json, gitignored)
> Project (.claude/settings.json, checked in) > User (~/.claude/settings.json)
```

Exception: `permissions` (allow/deny/ask) **accumulate** across all scopes — they don't follow override precedence like everything else.

Hooks Quick Reference

Hook	Fires	Use
<code>PreToolUse</code>	Before tool executes	Block/modify — exit code 2 to deny , exit 1 only warns
<code>PostToolUse</code>	After tool returns	Audit, side effects
<code>UserPromptSubmit</code>	Prompt sent	Inject context
<code>Stop</code>	Agent finishes	Validate result, persist state
<code>SubagentStart</code> / <code>SubagentStop</code>	Subagent lifecycle	Track parallel results
<code>PreCompact</code>	Before compaction	Archive full transcript

Hooks run in your process (no context-window cost). A `PreToolUse` deny enforces **even under `bypassPermissions`**. Keep hook scripts fast (~500ms guideline — they run synchronously in the loop).

Prompt Injection Defense (Indirect — the exam's favorite scenario)

- Untrusted content **only** in `tool_result` blocks, never `system` /plain `user` text.
- Label the content's nature/source in the tool description or result structure.
- State the untrusted-content policy explicitly in the system prompt.

- 4. JSON-encode untrusted strings (unambiguous delimiters, no "breakout").
- 5. Never put your own instructions inside a tool result — send them in the following `user` turn.
- 6. Screen tool outputs with a lightweight-model classifier before Claude acts on them.
- 7. Least privilege — bound the blast radius even if 1–6 all fail.

Structured Outputs

Mechanism	Constrains	Config
JSON outputs	The final response	<code>output_config.format</code> , <code>type: "json_schema"</code> , your <code>schema</code>
Strict tool use	Arguments passed to a tool	<code>strict: true</code> on the tool's <code>ToolParam</code>

Both use **constrained decoding** — invalid tokens literally can't be generated. Still check `stop_reason` (`refusal` / `max_tokens` can still break a "guaranteed" schema). First request on a new schema pays grammar-compile latency (cached 24h). Incompatible with message prefilling.

API Key / Secrets

- Shown **once** at creation (`sk-ant-...`) — capture to a secrets manager immediately, can't be retrieved again.
- Prefer short-lived federated credentials (Workload Identity Federation) over static keys where available.
- Rotate periodically, revoke immediately on suspected leak.

Claude Code Permission Modes (know all six)

Mode	Auto-approves	Isolated container only?
<code>default</code>	Reads only	No — baseline
<code>acceptEdits</code>	Reads, edits, common FS commands in working dir	No
<code>plan</code>	Reads only, no edits until plan approved	No
<code>auto</code>	Everything, classifier-reviewed	No — research preview
<code>dontAsk</code>	Only pre-approved + read-only; else auto- deny	No — CI/scripts
<code>bypassPermissions</code>	Everything, zero checks	Yes — never on a live workstation

`bypassPermissions` uniquely skips the protected-path guard the other modes keep. Deny rules always beat allow rules, at every mode, at every scope.

Human-in-the-Loop Placement

Destructive tool call about to run (write/delete/send)?	→ Gate: high risk, irreversible
Plan generated, about to execute?	→ Gate: medium risk, wrong plan = wrong outcome
Tool result has error flag / empty / out-of-bounds?	→ Gate: catches what retries won't

Agent Deployment: Three Wiring Paths

Need full control, or a constraint no library accommodates? → Raw Messages API loop
Want Claude Code's loop/context/tools without rebuilding, in-process? → Agent SDK
Long-running (min-hours), want a managed sandbox, no loop to build? → Managed Agents
▲ Managed Agents: server-side stateful sessions → NOT ZDR/HIPAA-BAA eligible.
PHI or ZDR requirement rules it out regardless of fit.

Deployment Platform Quick Pick

No cloud/residency constraint, want newest features first? → First-party Claude API
On AWS, want Anthropic's own model IDs/lifecycle? → Claude Platform on AWS
On AWS, want feature parity, compliance posture there? → Claude in Amazon Bedrock (Messages API)
Existing Bedrock integration, not yet migrated? → Claude on Bedrock (legacy, InvokeModel/Converse)
On Google Cloud, compliance posture there? → Google Vertex AI
Already running a product that embeds Claude? → Third-party (e.g. Microsoft Foundry)

Pin the **full model ID**, never a moving alias, in production. Retain the prior pinned version for rollback. Gate every promotion on the eval suite against a baseline score.

Regulated Data Constraints — What Each Rules Out

Constraint	Key rule
Attorney-client privilege	No consumer surface; firm-approved gateway with full logging (Anthropic doesn't log content by default)
HIPAA (PHI)	BAA-covered config only — BAA excludes Console/Workbench/beta/consumer plans
GDPR / EU residency	Direct API has no EU residency — route through Bedrock/Vertex with region pinned
FedRAMP / government	Only C4G, Bedrock GovCloud, or Vertex Assured Workloads — not AWS Marketplace Enterprise
Internal residency policy	Whatever cloud vendor is already approved — procurement rules out the rest

Eval Grading Method

One correct label/value, zero ambiguity? → Exact/string match (cheap, brittle)
Structured output (JSON/code/range)? → Code-graded check (format only, not content quality)
Open-ended quality (faithfulness, tone)? → LLM-as-judge (calibrate against human labels first)

Judge prompts: ask for strengths/weaknesses/reasoning **before** the score, or it drifts to a safe ~6 regardless of quality.

Tool Schema Description Pattern

Every tool description: **when to use it, AND when not to**. "Use this to find information" collides with any other retrieval tool. Add one exclusion sentence per tool to fix ambiguous routing — if that's not enough, merge the tools with a `type` param instead of lengthening descriptions further.

Batch API Numbers

- Up to **100,000 requests or 256MB** per batch call (whichever hits first).
- Results return in **arbitrary order** — use `custom_id` to match back to inputs.
- Chunking a loop over the *synchronous* endpoint is **not** batching — same rate limits, same per-request cost.

Image Token Cost

$\lceil \text{width}/28 \rceil \times \lceil \text{height}/28 \rceil$ visual tokens. A 1000×1000px image $\approx$ 1,296 tokens. Measure against real production images before shipping — resize is a 10-minute fix pre-deploy, expensive after.